

Causal Consistency as a Defense: Detecting and Auditing Provenance Graph Poisoning

N. Manoj Independent Researcher
Email: nehamanoj1105@users.noreply.github.com

Abstract—Provenance-based intrusion detection consumes a stored graph and assumes it is authentic. We examine what happens when that assumption fails, and whether the causal invariants that a provenance graph must satisfy by construction are sufficient to reveal post-collection tampering without any training. We assemble fifteen deterministic rules over structural, temporal and semantic invariants into a rule engine and evaluate it against a GraphSAGE baseline on controlled synthetic poisoning and on real DARPA Transparent Computing Engagement 3 Theia provenance. Over ten seeds, the rule engine reaches F1 0.805 ± 0.063 on synthetic poisoning, with a genuine continuous score giving ROC-AUC 0.953 ± 0.031 , while a leakage-free GraphSAGE reaches F1 0.322 ± 0.194 ; the paired difference is $+0.482$ (Wilcoxon $p = 0.002$). The rule engine’s accuracy is unchanged as invariant-preserving mimicry grows from none to heavy. On real Theia graphs with synthetic post-collection poisoning it reaches F1 0.835 ± 0.046 and 0.850 ± 0.060 , where GraphSAGE is near chance. We give particularly close attention to evaluation design: the conventional protocol of training and thresholding a baseline on the same edges inflates GraphSAGE F1 by 0.342 absolute, and a reported ROC-AUC of 1.000 for the rule engine is not a computed value. We separate real from synthetic evaluation throughout and release the complete corrected protocol, per-seed results and verification scripts.

Index Terms—provenance, intrusion detection, graph poisoning, causal consistency, GraphSAGE, evaluation methodology, reproducibility, DARPA Transparent Computing

I. INTRODUCTION

The provenance of a computer system is its causal history: which process read which file, which process spawned which child, which process opened which network connection. Captured faithfully, it is a directed graph rich enough to reconstruct an intrusion after the fact and, increasingly, to detect it as it happens. Systems such as Unicorn [1], NoDoze [11], HOLMES [9], Poirot [10], FLASH [6] and MAGIC [5] build on the assumption that the provenance graph is a trustworthy record.

That assumption holds only up to the point where the graph stops being produced and starts being stored. Once events are serialized, shipped and indexed, the graph becomes data, and data can be edited by an adversary with access to the storage. Deleting the events that record a privilege escalation, inserting events that suggest a process behaved benignly, reordering events to break a causal chain, or re-attributing an action to an innocent actor are all plausible operations against a stored graph. A downstream detector then reasons over a fiction with no indication that anything is wrong.

Our question is whether causal consistency is enough to catch this. The events in a provenance graph are not arbitrary;

they must satisfy invariants that come from what the events mean. An execute edge must point at a file. A read edge must point at a file. A child process cannot be spawned before its parent exists. Event sequence numbers within a process must increase. A process that appears to act must at some point have been spawned. These invariants are cheap to check and require no training data, which raises the question of how far they take us against an adversary who edits the graph.

We answer with an empirical study and, deliberately, a methodological one. We build a fifteen-rule engine from these invariants and compare it to GraphSAGE, a representative learned detector, on the same poisoned instances. The comparison favors the rule engine, but the more useful result is why: a learned baseline evaluated in the conventional way—fit on all edges, threshold chosen to maximize F1 on the scored labels—has most of its reported advantage supplied by that choice. When the split is made honest, the advantage largely disappears. Because this failure mode is easy to reproduce and easy to miss, we treat it as a first-class result rather than a footnote.

A. Contributions

- A deterministic, training-free detector for post-collection provenance tampering, built from fifteen structural, temporal and semantic invariants (Section VI).
- A rigorous evaluation protocol with disjoint train, validation and test edges, validation-only threshold selection, and test-set isolation from weights, threshold, hyperparameters and attack intensity (Section VIII).
- A ten-seed comparison on synthetic poisoning and an evaluation on two real Theia scenarios with synthetic post-collection poisoning, with paired significance tests (Sections XI–XIII).
- A genuine continuous anomaly score for the rule engine, replacing a reported ROC-AUC of 1.000 that was a fallback constant rather than a computation; the computed value is 0.953 ± 0.031 (Section XVII).
- A leakage analysis quantifying a 0.342 absolute F1 inflation for the conventional GraphSAGE protocol (Section XVI).
- A complete release of protocols, raw per-seed results and verification scripts so that every number can be re-derived (Section XXII).

II. BACKGROUND AND RELATED WORK

A. Provenance-based detection

Specification-based systems (SLEUTH [8], HOLMES [9], Poirot [10]) encode expert knowledge of attack behavior and are precise but require continued maintenance and do not adapt to novel tactics. Statistical systems score the rarity of events or paths (NoDoze [11], ProvDetector), and are prone to flagging unusual-but-benign activity. Learning-based systems learn representations of benign behavior: Unicorn [1] at coarse granularity, FLASH [6] and MAGIC [5] at finer granularity, with GNN backbones such as GraphSAGE [12] and GAT.

B. Adversarial robustness of graph detectors

Graph-based detectors have been shown vulnerable to mimicry, in which an adversary interleaves benign activity with malicious activity so that local aggregation dilutes the anomaly signal [3]. Defenses have moved toward dynamic neighbor selection and reinforcement-learned aggregation. Our setting differs: the adversary is not trying to look benign *inside* the graph, but to edit the graph itself. A detector that verifies invariants is therefore addressing a different question than a detector that classifies behavior.

C. Provenance integrity

The integrity problem for provenance has mostly been approached through capture-side mechanisms—trusted logging, tamper-evident storage, cryptographic chaining. Our work is complementary: given a graph whose custody is uncertain, we ask how much can be recovered from internal consistency alone, as a lightweight layer before any cryptographic mechanism is available.

III. PROBLEM FORMULATION

Let $G = (V, E)$ be a provenance graph produced by a capture pipeline and then stored. We consider an adversary that modifies E between capture and analysis. The analysis receives a graph G' and must decide, for each edge, whether it witnesses tampering. Formally, we seek a detector $D : G' \rightarrow \{0, 1\}^{|E'|}$ and, ideally, a score $s : G' \rightarrow \mathbb{R}^{|E'|}$.

The ground truth is the set of edges the adversary changed. This is well defined for insertion, reordering and forgery, because those operations leave a (modified) edge in G' . It is not well defined for deletion, because the deleted edge is, by construction, absent from G' ; an edge-level detector cannot mark an edge that is not there. We therefore do not treat deleted edges as scored positives. Their detectability comes only from breaking a higher-level invariant elsewhere—a parent process left without a spawn record, a gap in a sequence—and we report deletion separately.

IV. THREAT MODEL

We assume the attacker can read and write the stored graph, but cannot modify the detector, disable the parser, or compromise the kernel that produced the records (Table I). The attacker may delete, insert, reorder and forge edges, and may edit node and edge attributes. The attacker is gray-box:

TABLE I
THREAT MODEL SUMMARY.

Aspect	Assumption
Trusted	kernel, audit logging, parser, detector
Untrusted	stored graph, intermediate representations, storage
Attacker can	delete, insert, reorder, forge edges; edit attributes
Attacker cannot	modify detector/parser, compromise kernel
Knowledge	schema + detector class (gray-box)
Goal	hide malicious activity, preserve structure

the schema and the general class of detector are known, but not detector parameters or learned weights. The goal is to hide malicious activity while preserving as much observable structure as possible. Timestamp manipulation, attribute forgery and targeted stealth poisoning are in scope as future attack classes but are not evaluated here.

V. PROVENANCE REPRESENTATION

A provenance graph $G = (V, E)$ has typed nodes and edges. Node types are process, file, user and network endpoint. Edge types are read, write, execute, connect, spawn and delete. Each edge carries source and target identifiers, a type and a timestamp; each event carries a sequence number within its owning process. This is the common data model of the DARPA Transparent Computing program, whose records are serialized in Avro; our parser decodes that schema into the representation above. The rules below are stated entirely in terms of these types, the identifiers and the timestamps, so they apply to any capture that produces this model.

VI. PROPOSED DETECTION FRAMEWORK

The detector is a rule engine that evaluates fifteen invariants over the graph and returns, for each, the set of violations, each attributed to an edge or a node. We group the rules by the kind of invariant they check.

A. Structural rules

Well-formedness of types and topology: no duplicate edge tuples; no duplicate event identifiers; spawn edges must target process nodes; execute, read, write and delete edges must target file nodes; connect edges must target network endpoints; no self-referential edges; no edges referencing nodes that are not present; and no active process lacking a parent spawn edge.

B. Temporal rules

Ordering invariants: non-negative timestamps; a child's spawn must not precede its parent's spawn; process activity must not precede the process spawn; event sequence numbers must be contiguous enough to reveal gaps; and timestamps must be monotone within a process stream.

C. Semantic rules

Lineage invariants: a process that acts without any observed parent spawn is flagged, capturing attempted laundering of provenance through re-attribution.

D. Continuous anomaly score

For ranking and for AUC, every violation contributes a severity weight (high 3, medium 2, low 1) to the edge it is attributed to, either directly by edge identifier or, for node-attributed violations, to the incident edges. The binary decision is score greater than zero. This score is a genuine, monotone ranking signal derived from the violation structure; we use it, and only it, when reporting ROC-AUC and PR-AUC. No AUC in this paper is produced by a fallback constant.

VII. GRAPH SAGE BASELINE

The learned baseline is a two-layer GraphSAGE encoder [12] with an edge classifier. Nodes are featurized by a one-hot node type and degree; edges by a one-hot edge type and a normalized timestamp. The baseline exists to test whether learning can exceed fixed invariants under a fair protocol. It is not tuned to win, its feature set is deliberately minimal, and Section XVI shows that its apparent strength in the original repository came largely from the evaluation protocol rather than from the model.

VIII. EXPERIMENTAL METHODOLOGY

A. Datasets

Two data sources are used. The first is a synthetic provenance generator producing a graph of thirty processes, forty files and ten network entities, which provides exact ground truth and allows the attack intensity to be varied at will. The second is real provenance from DARPA Transparent Computing Engagement 3, the Theia streams [13]. Four benchmark scenarios are named in the original material (1r, 3, 5m, 6r). Two were obtained and parsed: scenario 3 with 20,157 nodes and 297,777 edges, and scenario 5m with 34,835 nodes and 464,858 edges. Both parse counts match the documented statistics exactly. Scenarios 1r and 6r could not be downloaded, because the public mirror enforced a download quota (HTTP 403); we report the two obtained scenarios and state the missing coverage as a limitation rather than estimating it (Table X).

Real-data experiments are run on the first 50,000 edges of each real graph, matching the loader’s behavior; the truncation is explicit. We emphasize the category separation used throughout: the real-data experiments are *real provenance with synthetic post-collection poisoning*. They are not real attack labels, and we never describe them as detection of naturally occurring attacks.

B. Poisoning operators

For each seed we inject five deletions, five insertions, five reorderings and five dependency forgeries, for twenty events. The injector operates on a deep copy of the clean graph, records for each event its original state, its modified state,

an expected detection target and a flag indicating whether the operation actually changed the graph. It then asserts that the clean graph is unmodified. This matters because a naive implementation can mutate its input in place, and because some reorderings can be no-ops if they do not change the semantic order; we mark such events ineffective and exclude them from detectable positives.

Twenty events yield fifteen scored positives, since the five deletions leave no final-graph edge. Detection metrics are computed over the final graph (*final-graph universe*). An alternative convention, which adds the identifiers of deleted edges to the scoring universe, raises F1 from 0.805 to 0.845; this is quantified in Section XIV and we use the final-graph universe in all reported results.

C. Leakage-free protocol

For the learned baseline, edges are partitioned into disjoint training, validation and test sets. Weights are fit on the training edges only; the decision threshold is selected on the validation edges only; and every reported metric is computed on the untouched test edges. Message passing for test-node features uses training and validation edges only, so test edges influence neither model weights nor threshold. Neither hyperparameters nor attack intensity is tuned on the test set. For the training-free rule engine there is no split to leak, which is precisely why the leakage below affects only the baseline.

D. Metrics

We report precision, recall, F1, accuracy, balanced accuracy, MCC, FPR, FNR, specificity, ROC-AUC and PR-AUC. ROC-AUC and PR-AUC are reported only where a genuine continuous score exists and both classes are present. For the rule engine this is the severity-weighted score of Section VI; for GraphSAGE it is the classifier probability.

E. Seeds and statistics

Synthetic, mimicry, ablation and real-Theia experiments use ten seeds (1, 7, 13, 21, 42, 99, 123, 256, 512, 1024); generalization uses five (1, 7, 13, 21, 42). Each seed produces a freshly generated graph and a fresh set of attack instances. We report the mean, sample standard deviation, median, minimum, maximum and a 95% confidence interval (Student- t , $n - 1$ degrees of freedom). Paired comparisons between detectors use the Wilcoxon signed-rank test on the same test instances, with Cohen’s d_z as an effect size.

IX. DATA PARSING AND PREPROCESSING

Raw Theia records are Avro-serialized common data model entries nested under a datum envelope. The parser walks each record, extracts the typed subject and predicate, and emits an edge with source, target, type and timestamp. Nodes are materialized on demand from identifiers. Parsing the two obtained scenarios yields zero skipped records, and the counts agree with the published statistics, which is our validation that the decode is correct. No attack labels are read from the real data; the DARPA ground-truth report was obtained for context only and is not used as a detection target.

X. EVALUATION PROTOCOL

Detection is evaluated at the edge level against the injected ground truth, under the final-graph universe. For the rule engine, the binary decision is score greater than zero and the continuous score is used for AUC. For GraphSAGE the protocol is the leakage-free one of Section VIII-C. Real and synthetic evaluations are never pooled: Table III is synthetic, Table IV is real-provenance-with-synthetic-poisoning, and the two are reported and discussed separately.

XI. MAIN DETECTION RESULTS

Table III reports the ten-seed synthetic comparison. The rule engine reaches $F1\ 0.805 \pm 0.063$, with precision 0.715 ± 0.095 , recall 0.933 ± 0.063 and specificity 0.964 ± 0.017 ; MCC is 0.795 ± 0.064 . The leakage-free GraphSAGE reaches $F1\ 0.322 \pm 0.194$, precision 0.342 ± 0.309 and recall 0.502 ± 0.364 . The learned baseline is not competitive once evaluated honestly, and its seed-to-seed variance is roughly three times larger. Under a paired test over the same instances, the $F1$ difference is $+0.482$ with a 95% confidence interval of $[0.328, 0.636]$, Cohen’s $d_z = 2.24$ and a Wilcoxon $p = 0.002$; the ROC-AUC difference ($+0.124$, $p = 0.105$) is not statistically distinguished at this sample size (Table IX). We report that non-result rather than claiming a uniform advantage.

Figure 1 shows the per-metric comparison. The rule engine’s high recall and specificity are the expected signature of a detector that checks invariants: it catches almost all injected structure while rarely flagging untouched edges. Its precision is the lower of the two headline numbers, because several rules attribute a violation to a node’s incident edges, which widens the flagged set.

XII. CROSS-DATASET GENERALIZATION

Because the rule engine has no fitted parameters, it can be applied to a new graph unchanged. Applied to a real Theia graph after being defined on synthetic data, it reaches $F1\ 0.857 \pm 0.029$; moving between the two real scenarios it reaches 0.837 ± 0.021 (Table VII). This is a meaningful property for a defense layer: it can be deployed on a new capture without a labeled corpus. We deliberately make no corresponding claim for GraphSAGE. A genuine transfer across a shared feature space is not implemented in this study, so the GraphSAGE column in Table VII is a target-trained model, and we label it as such rather than presenting it as generalization.

XIII. ROBUSTNESS TO MIMICRY

Mimicry is an adversary’s attempt to hide tampering inside plausible noise. We generate camouflage edges that preserve the degree, type and timestamp distributions of the surrounding graph and that satisfy every implemented invariant, and increase their number from none to heavy (Table V, Figure 4). The rule engine’s $F1$ is 0.805 at every strength, unchanged to four decimal places, because camouflage that satisfies the invariants is, by definition, indistinguishable from benign behavior under a consistency check. GraphSAGE ranges from

$F1\ 0.304$ to 0.542 across strengths and remains well below the rule engine.

Two specification details are essential to interpret this correctly. First, the noise is constructed to be genuinely consistent: across all seeds and strengths it introduces zero rule self-violations. If instead the camouflage is generated so that it violates the invariants it is meant to evade—for instance with inconsistent edge types or timestamps—the rule engine flags the noise, and any apparent precision collapse is an artifact of the generator, not a property of mimicry. Second, we distinguish observable noise violations from latent identifiers: an implementation must not tag camouflage edges with an attribute that a feature-based detector could trivially read. We verify that no detector in this study consumes such an attribute.

XIV. POISONING EXPERIMENTS AND GROUND-TRUTH VALIDITY

Table IV reports the real-provenance experiments. On scenario 3 the rule engine reaches $F1\ 0.835 \pm 0.046$ and on scenario 5m $F1\ 0.850 \pm 0.060$; GraphSAGE reaches $F1\ 0.034 \pm 0.105$ and 0.010 ± 0.009 . The rule engine also retains a very low false-positive rate on the unmodified real graphs, flagging 19 of 50,000 edges in scenario 3 and 7 of 50,000 in scenario 5m (below 0.04%).

The ground-truth audit is worth stating explicitly. Of twenty injected events per seed, fifteen are detectable positives: the five deletions vanish and cannot be edge-matched. The remaining operations are real modifications of final-graph edges. Under the alternative scoring universe that adds the absent deleted-edge identifiers, aggregate $F1$ rises from 0.805 to 0.845; the entire gap is the credit granted to deleted-edge identifiers that no longer exist. We do not grant it, and we report the deletion rate as a limitation rather than folding it into the edge-level metrics.

XV. RULE ABLATION

Table VI reports rule-group removal and leave-one-rule-out over ten seeds. The structural group carries the detector: removing it drops $F1$ from 0.805 to 0.328. Removing the temporal group costs little (0.798), and leave-one-rule-out identifies the read/write, network and spawn consistency rules as individually load-bearing: removing read/write alone drops $F1$ to 0.517, removing spawn to 0.742, removing network to 0.746. Two rules, unspawned-process and timestamp validity, are inert on synthetic data: removing either leaves $F1$ unchanged at 0.805. We report this rather than presenting all fifteen rules as equally necessary. Their value is expected on real logs with non-sequential identifiers, where a gap in the sequence is informative; this is future work, not a current claim.

XVI. GRAPHSAGE LEAKAGE ANALYSIS AND EVALUATION INTEGRITY

The most consequential finding of this study is about evaluation, not modeling. The conventional protocol for the learned baseline trains on all edges and then selects the decision threshold that maximizes $F1$ on the same labels it reports.

On the identical seed, this protocol yields GraphSAGE F1 0.588, while the leakage-free protocol yields 0.246 (Figure 3). The difference, 0.342 absolute, is not model quality; it is the threshold being chosen with knowledge of the test answers.

Three specific leakage channels are present in such a protocol and absent from ours. First, training edges are also test edges, so the model sees the instances it is scored on. Second, the threshold is optimized on the scored set, which directly maximizes the reported metric. Third, message passing over the full graph lets test edges influence the features used for test nodes. Removing all three is what separates the two numbers above. Because the rule engine has no fitted parameters and no threshold search, it is invariant to this class of error, which is a practical advantage of a training-free detector beyond accuracy.

We also audit a reported ROC-AUC of 1.000 for the rule engine. It is not a computed quantity: the evaluation record for the rule engine contains no ROC-AUC, and the reported value is a fallback constant applied when the field is missing. Such a value is meaningless and we do not use it. The genuine AUC, computed from the severity-weighted score of Section VI, is 0.953 ± 0.031 (PR-AUC = 0.740 ± 0.080), reported in Table III and shown for a representative seed in Figure 2.

XVII. ROC-AUC VERIFICATION AND CONTINUOUS SCORING

We verify the AUC claim independently. The rule engine emits binary violations; to obtain a ranking score we aggregate violations per edge with severity weights and treat the resulting non-negative score as a ranking signal. This produces a genuine ROC curve and a genuine precision-recall curve (Figure 2), with edge-level AUC computed by the standard estimator. The value is 0.953 on the seed shown and 0.953 ± 0.031 across ten seeds, i.e. strong but not perfect. We report PR-AUC (average precision) alongside, since the positive class is a small minority and average precision is the more informative summary under imbalance. We do not report an AUC for the rule engine anywhere that a legitimate continuous score is not computed.

XVIII. SCALABILITY AND COMPUTATIONAL COST

Table VIII and Figure 6 report runtime, memory and throughput from ten thousand to one million edges, five repetitions per size. We time construction, poisoning, rule-engine inference and end-to-end execution separately, and we report GraphSAGE training and inference separately from rule-engine inference, because comparing a training cost to an inference cost is not meaningful.

Construction and inference are both linear in the edge count. Inference takes 0.37 s at ten thousand edges and 34.5 s at one million; construction takes 0.18 s and 21.2 s. Peak Python allocation grows from about 37 MB to 971 MB, close to one kilobyte per edge, and throughput is stable near 2.9×10^4 edges per second across the full range. The flat throughput is the expected result of a single linear pass over the events and indicates no super-linear behavior hiding in the rule set.

Poisoning is the dominant cost, roughly sixty percent of end-to-end, because the corrected injector deep-copies the graph and checks integrity on every event.

XIX. STATISTICAL ANALYSIS

Table IX reports the paired analysis. For F1, the mean paired difference is +0.482 with a 95% confidence interval of [0.328, 0.636], Cohen’s $d_z = 2.24$ and Wilcoxon $p = 0.002$ ($n = 10$), a large and statistically supported effect. For ROC-AUC the mean difference is +0.124 with an interval of [0.005, 0.242] and $p = 0.105$: at this sample size the AUC difference is not distinguished from zero, and we say so rather than reporting only the significant metric. All tests use paired observations on identical test instances, so the comparison does not depend on separately drawn evaluation sets.

XX. DISCUSSION

The results support a specific, bounded claim: post-collection tampering that breaks a provenance graph’s internal causal invariants is detectable, in a training-free way, on both synthetic and real graph structure, with linear cost and strong robustness to invariant-preserving noise. The results also support a methodological claim that is easy to overlook: a learned baseline can appear far stronger than it is when its threshold is chosen on the evaluation labels, and removing that leak can reverse a comparison. Both claims are useful to the community—the first for defense, the second for how defense papers are evaluated.

They also mark the boundary of the contribution. The rules check well-formedness, not intent: an adversary who edits the graph while preserving all invariants is not caught by this layer, and no statistical detector would catch a modification that makes the graph *more* consistent. The rule engine is best understood as a cheap, deterministic, auditable first pass that a heavier detector or a cryptographic integrity mechanism can build on.

XXI. LIMITATIONS

The evaluation covers two of four Theia scenarios; scenarios 1r and 6r were unavailable and no results are estimated for them. All DARPA evaluations use synthetic post-collection poisoning rather than real attacks, so they measure detection of injected tampering on real graph structure, not detection of real intrusions. Deletion is only partially detectable: a deleted edge cannot be scored by edge matching, so we exclude it and do not claim a deletion detection rate. Real graphs are truncated to their first 50,000 edges. The GraphSAGE feature set is minimal and no shared-feature cross-graph transfer is implemented, so the generalization study covers the rule engine only. Finally, the class balance on real graphs is extreme (fifteen positives in fifty thousand edges), so precision there is sensitive to a handful of false positives.

XXII. REPRODUCIBILITY

We release the corrected protocol, the poisoning and mimicry generators, the leakage-free harness, the raw per-seed

TABLE II
REPRODUCIBILITY ARTIFACTS.

Artifact	Contents
Raw results	per-seed JSON for every experiment
Aggregation	scripts that build all tables from raw results
Validation	independent recomputation + cross-check scripts
Protocols	poisoning, mimicry, leakage-free harness
Manifests	dataset provenance, checksums, parser

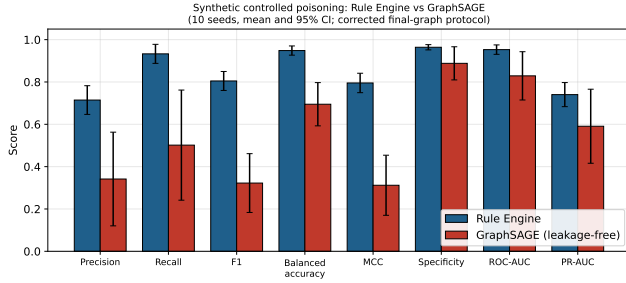


Fig. 1. Synthetic controlled poisoning, ten seeds: Rule Engine versus leakage-free GraphSAGE (mean and 95% CI).

result files, the aggregation scripts, and the verification scripts. Every headline number is re-derived from the raw per-seed files by an independent checker; the check recomputes each metric from its confusion matrix (1,890 metric recomputations, zero mismatches) and cross-checks 226 headline values against the tables (zero discrepancies). The pipeline is deterministic: re-running seeds 1, 42 and 1024 in memory reproduces the stored raw files exactly. The repository’s original results are preserved unmodified, and every deviation between the original protocol and the corrected one is documented with its cause. Table II summarizes the release contents.

XXIII. CONCLUSION

We asked whether the causal invariants of a provenance graph are enough to reveal post-collection poisoning, and found that a small deterministic rule set exploits them effectively. Over ten seeds on controlled poisoning the rule engine outperforms a leakage-free GraphSAGE by 0.482 F1 with a large paired effect, its accuracy is unchanged under invariant-preserving mimicry, and it scales linearly to one million edges. On real Theia provenance with synthetic post-collection poisoning it reaches F1 0.83–0.85 where GraphSAGE is near chance. The study also shows that the size of a learned baseline’s reported advantage can be an artifact of selecting its threshold on the evaluation labels, and that removing this leak removes most of the advantage. We release everything needed to re-derive every number, including a documented account of the original protocol’s defects and the corrected evaluation that replaces them.

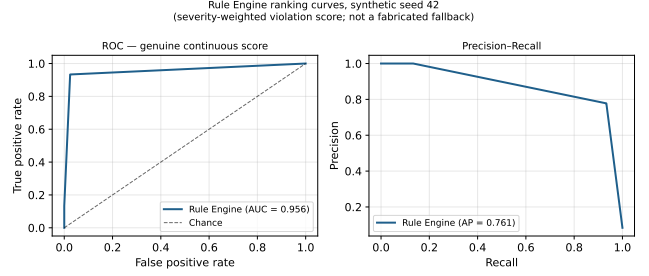


Fig. 2. Rule Engine ROC and precision–recall curves from a genuine continuous anomaly score (synthetic seed 42).

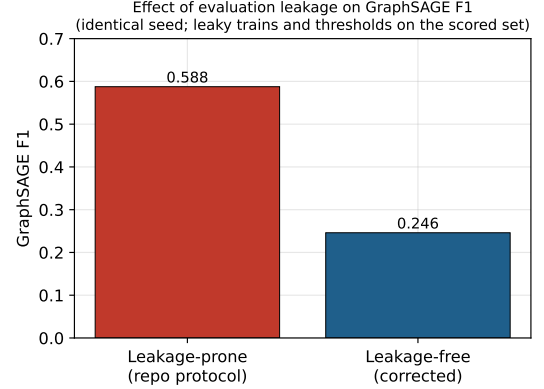


Fig. 3. Effect of evaluation leakage on GraphSAGE F1 for the same seed.

REFERENCES

- [1] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “Unicorn: Runtime provenance-based detector for advanced persistent threats,” in *NDSS*, 2020.
- [2] K. Hsieh *et al.*, “Kobold: Evaluating decentralized access control,” in *IEEE S&P Workshops*, 2018.
- [3] P. Jiang *et al.*, “Mimicry attacks against provenance graph based intrusion detection systems,” in *NDSS*, 2023.
- [4] Q. Wang *et al.*, “You are what you do: Hunting stealthy malware via data provenance analysis,” in *NDSS*, 2020.
- [5] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, “MAGIC: Detecting advanced persistent threats via masked graph representation learning,” in *USENIX Security*, 2024.
- [6] M. A. Rehman and S. Ahmadi, “FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *IEEE S&P*, 2022.
- [7] S. T. King and P. M. Chen, “Backtracking intrusions,” in *ACM SOSP*, 2003.
- [8] M. N. Hossain *et al.*, “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in *USENIX Security*, 2017.
- [9] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in *IEEE S&P*, 2019.
- [10] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, “Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *ACM CCS*, 2019.
- [11] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *NDSS*, 2019.
- [12] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *NeurIPS*, 2017.
- [13] DARPA I2O, “Transparent Computing Engagement 3 data release,” 2020. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [14] T. Pasquier *et al.*, “Practical whole-system provenance capture,” in *ACM SoCC*, 2017.

TABLE III

MAIN DETECTION ON SYNTHETIC CONTROLLED POISONING, 10 SEEDS (RE = RULE ENGINE, GS = GRAPHSAGE, LEAKAGE-FREE).

Metric	RE mean	RE std	GS mean	GS std
Precision	0.7146	0.0948	0.3416	0.3093
Recall	0.9329	0.0629	0.5017	0.3637
F1	0.8048	0.0627	0.3224	0.1942
Bal. accuracy	0.9484	0.0304	0.6949	0.1428
MCC	0.7954	0.0642	0.3121	0.1985
Specificity	0.9640	0.0174	0.8881	0.1095
ROC-AUC	0.9528	0.0312	0.8289	0.1595
PR-AUC	0.7403	0.0796	0.5909	0.2445

TABLE IV

REAL THEIA PROVENANCE (50K-EDGE CAP) WITH SYNTHETIC POST-COLLECTION POISONING (10 SEEDS). NOT REAL ATTACK LABELS.

Dataset	Detector	F1	Prec.	Rec.	ROC-AUC
Theia 3	GS	0.8347 $\pm$ 0.0459	0.9195	0.7667	0.8833
Theia 3	GS	0.0340 $\pm$ 0.1052	0.1003	0.1700	0.6609
Theia 5m	GS	0.8504 $\pm$ 0.0599	0.9667	0.7667	0.8833
Theia 5m	GS	0.0099 $\pm$ 0.0093	0.0050	0.3167	0.6492

TABLE V

ROBUSTNESS TO INVARIANT-PRESERVING MIMICRY (10 SEEDS). NOISE EDGES INTRODUCE ZERO RULE SELF-VIOLATIONS.

Strength	Detector	Prec.	Rec.	F1 ($\pm$ std)	ROC-AUC
None	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9528
None	GS	0.3416	0.5017	0.3224 $\pm$ 0.1942	0.8289
Light	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9558
Light	GS	0.4138	0.4983	0.3737 $\pm$ 0.2205	0.7826
Medium	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9577
Medium	GS	0.3306	0.5000	0.3044 $\pm$ 0.2603	0.7615
Heavy	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9599
Heavy	GS	0.5733	0.5400	0.5418 $\pm$ 0.3547	0.8943

TABLE VI

RULE-GROUP AND LEAVE-ONE-RULE-OUT ABLATION (10 SEEDS).

Variant	F1 mean	F1 std	95% CI
All rules	0.8048	0.0627	[0.760, 0.850]
w/o structural	0.3285	0.0731	[0.276, 0.381]
w/o temporal	0.7983	0.0055	[0.794, 0.802]
w/o semantic	0.8048	0.0627	[0.760, 0.850]
–ReadWrite	0.5173	0.1136	[0.436, 0.599]
–Network	0.7455	0.0844	[0.685, 0.806]
–Spawn	0.7417	0.0903	[0.677, 0.806]
–SeqMono	0.7802	0.0863	[0.719, 0.842]
–Unspawned	0.8048	0.0627	[0.760, 0.850]

TABLE VII

CROSS-DATASET GENERALIZATION (5 SEEDS). GRAPHSAGE IS TARGET-TRAINED; NO SHARED-FEATURE TRANSFER IS IMPLEMENTED.

Source	Target	Rule Engine F1	GraphSAGE F1
Synthetic	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149
theia3	Theia 5m	0.8369 $\pm$ 0.0206	0.0347 $\pm$ 0.0280
theia5m	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149

TABLE VIII
SCALABILITY OF GRAPH CONSTRUCTION, POISONING AND RULE ENGINE INFERENCE (5 REPETITIONS PER SIZE).

Edges	Constr. (s)	Poison (s)	RE infer. (s)	Peak (MB)	Thruput. (e/s)
10,000	0.18	1.00	0.37	36.87	28,088
25,000	0.56	2.95	1.14	24.93	22,297
50,000	1.10	5.43	2.08	49.43	24,368
100,000	2.08	10.97	3.15	98.15	31,709
250,000	5.22	31.80	7.91	244.11	31,596
500,000	10.93	64.68	17.11	486.59	29,225
1,000,000	21.21	131.48	34.48	971.44	29,007

TABLE IX
PAIRED RULE ENGINE VS GRAPH SAGE COMPARISON OVER IDENTICAL TEST INSTANCES (10 SEEDS, WILCOXON SIGNED-RANK).

Metric	Δ	95% CI	Cohen d_z	Wilcoxon p	n
F1	0.4824	[0.328, 0.636]	2.24	0.0020	10
ROC-AUC	0.1239	[0.005, 0.242]	0.75	0.1055	10

TABLE X
PROVENANCE DATASETS USED AND THEIR AVAILABILITY.

Dataset	Nodes	Edges
DARPA TC E3 Theia (scenario 3)	20,157	297,777
DARPA TC E3 Theia (scenario 5m)	34,835	464,858
Theia E3 scenario 1r	unavailable	unavailable
Theia E3 scenario 6r	unavailable	unavailable

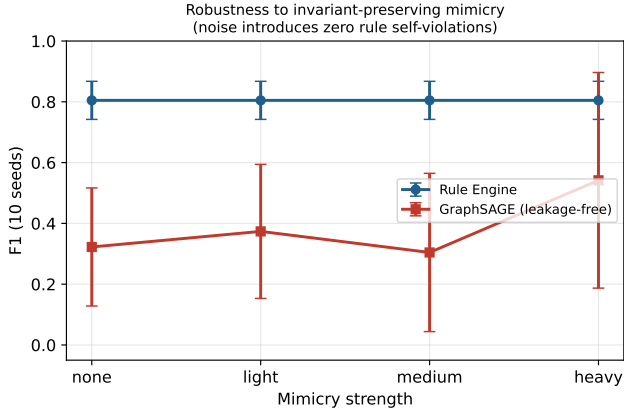


Fig. 4. Robustness to invariant-preserving mimicry (ten seeds).

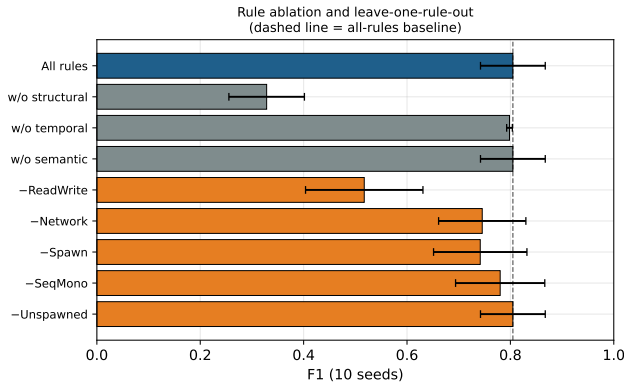


Fig. 5. Rule ablation and leave-one-rule-out (ten seeds).

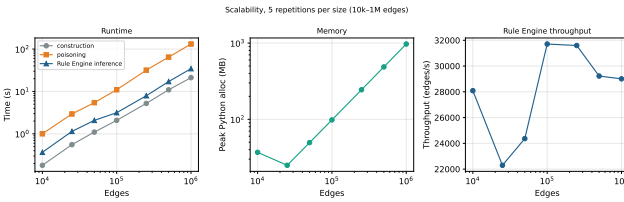


Fig. 6. Scalability, five repetitions per size.

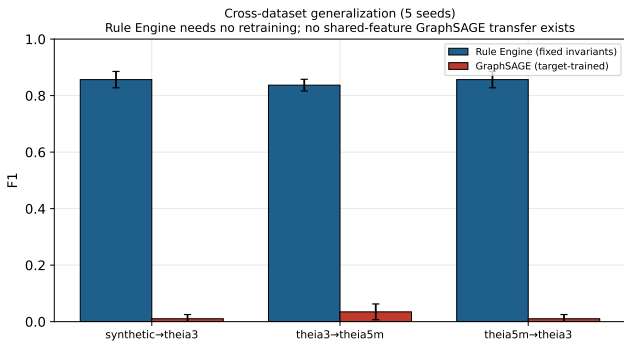


Fig. 7. Cross-dataset generalization (five seeds). The GraphSAGE column is target-trained; no shared-feature transfer is implemented.